

Image Classification and Few-Shot Learning on the CINIC-10 Dataset

Karolina Dunal, Mateusz Deptuch, Zofia Kaminska

March 2026

Abstract

This work investigates multiclass image classification on the CINIC-10 benchmark dataset. The study systematically evaluates and compares three prominent convolutional neural network (CNN) architectures: ResNet-50, EfficientNet-B4, and Xception. The research is divided into three main experimental phases: first, identifying optimal hyperparameter configurations under standard supervised training; second, investigating few-shot learning by adapting ImageNet-pretrained models using a supervised contrastive objective exploring various data augmentation strategies, followed by evaluation within a prototypical network framework; and finally, assessing the robustness and performance gains of model ensembling through hard and soft voting strategies across various combinations of models and training paradigms.

Contents

1	Introduction	2
1.1	Research Problem Description	2
1.2	Literature Review	3
1.2.1	Benchmark Datasets	3
1.2.2	CNN Architectures	3
1.2.3	Few-Shot Learning and Contrastive Pretraining	3
2	Experimental Setup	4
2.1	Dataset	4
2.1.1	Technical Specifications	4
2.2	Model Architectures	4
2.2.1	ResNet-50	4
2.2.2	EfficientNet-B4	5
2.2.3	Xception	6
2.3	Experiments – Part 1: Hyperparameter Tuning	6
2.4	Experiments – Part 2: Few-Shot Learning with Contrastive Pretraining	7
2.4.1	Method Description	7
2.4.2	Planned Experiments	8
2.5	Experiments – Part 3: Model Ensembling	9
3	Results Analysis	9
3.1	Individual Model Performance	9
3.2	Few-Shot Learning Results	13
3.3	Ensemble Results	16
4	Conclusions	20
5	Appendix	22

1 Introduction

1.1 Research Problem Description

Image classification is one of the most fundamental tasks in computer vision. It consists of assigning a semantic label to an input image drawn from a set of predefined categories. When dealing with only two classes, the problem is referred to as binary classification. When the number of classes increases, it becomes a multiclass classification problem. There are additional challenges that come with a higher number of classes: the decision boundary becomes more complex, as the model must learn discriminative features across all classes at once; the probability of having similar-looking categories also increases with the number of classes, which may significantly affect the difficulty of the task.

This work focuses on multiclass image classification on the CINIC-10 dataset. CINIC-10 (CINIC-10 Is Not ImageNet or CIFAR-10) is a benchmark image dataset containing 270,000 images distributed across 10 object classes. A detailed description of the dataset, is provided in Sections 1.2.1 and 2.1.

There are some classical machine learning approaches to image classification, such as support vector machines (SVMs) and random forests. However, they remain highly limited because they rely on self-defined feature representations. While effective in constrained settings, such methods struggle to capture complex patterns, textures and spatial relations present in natural images. Therefore, deep learning, specifically convolutional neural networks (CNNs), have become dominant in solving for image classification tasks. Those architectures learn complex feature representations directly from raw pixel data through successive layers of multiple learnable filters, pooling operations, and non-linear activations. More recently, transformer-based architectures such as Vision Transformers (ViT) and hybrid CNN-transformer models have achieved state-of-the-art results on many benchmarks, though they typically require large amounts of training data and significant computational resources to match well-tuned CNNs.

Three architectures were selected to be examined in this work: ResNet-50, EfficientNet-B4, and Xception. Each one represents a different design philosophy making them interesting candidates for comparative study on a challenging benchmark. Training those deep neural networks is a non-trivial optimisation problem. The space of hyperparameters, including learning rate, batch size, optimiser choice, regularisation strength, and dropout rate, is large. Small changes in any of these can significantly impact generalisation performance. In this work, the impact of these factors on classification accuracy is explored. Multiple data augmentation techniques are also considered. By artificially expanding the training set through label-preserving transformations such as random flips, rotations, colour jitter, and noise injection, we hope to improve accuracy of the selected models. Additionally, a custom fragment-wise approach implemented via the Augmentor library is investigated.

A complementary strategy to standard supervised training is few-shot learning (FSL), which aims to build classifiers capable of generalising to new classes or new data distributions from only a small number of labelled examples. FSL is particularly relevant in domains where labelled data is unavailable or expensive to obtain. Contrastive learning has emerged as a powerful pretraining approach that can improve the quality of learned representations, making models better suited to the few-shot learning paradigm. In our setting, we adapt ImageNet-pretrained models using a contrastive objective. We treat augmented versions of images from the same class as positive pairs and images from different classes as negative pairs, before employing the resulting embeddings within a prototypical network framework for classification.

Finally, we also implement and analyse two model ensembling methods. By combining the predictions of multiple independently trained models, we expect to improve classification accuracy. Both hard voting (majority vote across class predictions) and soft voting (averaging predicted class probabilities) are evaluated for multiple combinations of the three architectures within both training paradigms (standard and few-shot).

1.2 Literature Review

1.2.1 Benchmark Datasets

The three datasets most relevant to this project are CIFAR-10, ImageNet, and CINIC-10. CIFAR-10 [8] was introduced in 2009 by Krizhevsky et al. as a standard benchmark for evaluating image recognition algorithms on small images. It contains 60,000 colour images of resolution 32×32 pixels, distributed equally across 10 object classes. Its relatively small image size and moderate dataset size make it computationally accessible while still challenging, therefore it has remained a reference benchmark over the years.

ImageNet [4] is a much larger dataset introduced by Deng et al. (2009) as part of the ImageNet Large Scale Visual Recognition Challenge. It contains over a million labeled training images of a 1,000 classes at high resolution (224×224 pixels or larger). ImageNet has been the primary benchmark driving advances in deep CNN architectures, and pretrained ImageNet weights are most often used as a starting point for transfer learning on smaller datasets.

CINIC-10 [3] was created in 2018 by Darlow et al. as a more challenging replacement for CIFAR-10. It was constructed by combining all 60,000 original CIFAR-10 images with 210,000 images from the ImageNet dataset that were downsized and filtered to match the 10 CIFAR-10 classes. Crucially, CINIC-10 retains the small image resolution of CIFAR-10 (32×32) while incorporating the greater visual diversity of ImageNet, exposing models to distribution shift within each class. More technical details are provided in Section 2.1.

1.2.2 CNN Architectures

Convolutional neural networks were first introduced by LeCun et al. [9] (1998) in the context of handwritten digit recognition (LeNet). A CNN applies convolutional filters that detect spatially local patterns, followed by pooling operations that reduce spatial dimensionality. These operations are repeated across multiple layers, enabling the network to build increasingly abstract representations from low-level edges and textures to high-level semantic concepts.

ResNet-50 [5] (He et al., 2016) introduced residual learning as a solution to a problem of vanishing gradients that occurs when training very deep networks. Residual blocks introduce shortcut connections that skip one or more layers, allowing the gradient to flow directly from later layers back to earlier ones.

Standard convolutions learn spatial correlations and cross-channel correlations simultaneously; depthwise separable convolutions proposed by Xception [1] (Chollet, 2017) instead apply a separate convolution for each input channel and then combine the results using a 1×1 pointwise convolution. This decomposition reduces computational cost while maintaining representational power.

EfficientNet-B4 [11] (Tan & Le, 2019) is based on a study of how to scale CNNs along three dimensions—number of layers, number of channels, and input resolution—simultaneously and in an organised manner. These three scaling dimensions are interdependent: a larger resolution image benefits from a deeper network to capture the finer-grained patterns.

1.2.3 Few-Shot Learning and Contrastive Pretraining

Few-shot learning (FSL) aims to classify images using minimal labeled examples through metric-, model-, or optimization-based methods [6], of which commonly known are transfer learning and contrastive learning. A prominent metric-based approach is Prototypical Networks [10], which classify query images based on their proximity to a calculated mean prototype vector for each class. In our work we integrate a supervised contrastive learning objective [7] as a preceding training step. By grouping same-class images as positive pairs and distancing cross-class images as negative pairs in the embedding space, contrastive pretraining provides rich, generalisable features that naturally complement the few shot learning setup.

2 Experimental Setup

AI Assistance Note: GitHub Copilot’s coding assistance was utilised during the implementation of the experimental framework.

Computational Limitations Note: Due to computational limitations, we prioritized the breadth of our evaluation over conducting multiple runs per configuration. Therefore, the findings presented here serve to identify key trends rather than to show statistical significance. To reduce the resources necessary for each run, we adapted the pipeline by adjusting network architectures to process 32×32 images (Part 1) and by reducing the validation and test sets to a representative 10% in subsequent phases (Parts 2 and 3).

2.1 Dataset

All experiments in this work use the CINIC-10 dataset, which is publicly available on Kaggle. The dataset and its associated benchmark results are described in Darlow et al. (2018) [3].

2.1.1 Technical Specifications

The key properties of CINIC-10 are summarised below:

- Image resolution: 32×32 pixels (identical to CIFAR-10).
- Image format: PNG with 3 colour channels (RGB).
- Number of classes: 10, balanced (27,000 images per class).
- Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- Data splits: train, validation, test – 90,000 images each.
- Composition: CIFAR-10 images and ImageNet images downsampled to 32×32 , filtered to the same 10 classes.

No preprocessing steps were applied beyond the normalisation and augmentation procedures. Because the dataset is perfectly balanced across classes, standard accuracy serves as the primary evaluation metric. Sample images with corresponding labels can be found in Figure 1 below.

2.2 Model Architectures

Three CNN architectures were selected for this study on the basis of their architectural diversity, their widespread use in the literature, and their availability in standard deep learning libraries. All three are implemented in PyTorch and adapted to accept the 32×32 input resolution of CINIC-10. Table ?? summarises the key properties of each model.

Model	Parameters	Type	Key Feature
ResNet-50	~25.6M	CNN	Residual connections; bottleneck blocks; batch normalisation
EfficientNet-B4	~19M	CNN	Compound scaling; MBConv blocks; SE attention
Xception	~23M	CNN	Fully depthwise separable convolutions; residual connections;

Table 1: Summary of investigated architectures.

2.2.1 ResNet-50

ResNet-50 consists of 50 layers organised into four stages of residual bottleneck blocks, preceded by a convolutional stem and a max-pooling layer. Each bottleneck block applies a 1×1 convolution to reduce channel dimensionality, a 3×3 convolution, and a 1×1 convolution to restore it, with a shortcut connection added around the triple-convolution block. The final stage is followed by global average pooling and a



Figure 1: Representative samples from the CINIC-10 dataset. Each image is labeled with one of the ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, or truck.

fully connected classification head. The architecture is loaded via PyTorch’s torchvision library (torchvision.models.resnet50) with pretrained weights disabled (pretrained=False), allowing the model to be trained from scratch.

The standard ResNet-50 is designed for 224×224 images: the opening 7×7 convolution (stride 2) followed by a 3×3 max-pooling layer (stride 2) together downsample the spatial dimensions by a factor of 4 before the residual blocks are reached, which would reduce a 32×32 input to just 8×8 pixels, which would be too small for the following layers. To adapt the architecture to the CINIC-10 resolution, two modifications were made in the implementation:

- The first convolutional layer was replaced with a 3×3 convolution (stride 1, padding 1, 64 output channels, no bias), preserving the 32×32 spatial resolution while keeping the same number of output feature maps.
- The max-pooling layer was replaced with `nn.Identity()`, fully removing the additional stride-2 down-sampling step.

A dropout layer (default rate 0.5, configurable) is inserted immediately before the final fully connected layer to act as a regulariser during training. The classification head outputs logits for the 10 CINIC-10 classes.

2.2.2 EfficientNet-B4

EfficientNet-B4 is the fourth variant in the EfficientNet family, scaled from the EfficientNet-B0 baseline using a compound coefficient of $\phi = 4$.

The core building block is the Mobile Inverted Bottleneck Convolution (MBConv) with Squeeze-and-Excitation (SE) channel attention. EfficientNet-B4 has approximately 19 million parameters. In this implementation, the architecture is loaded from torchvision (pretrained=False).

The standard EfficientNet-B4 stem (`features[0][0]`) is a 3×3 convolution with stride 2 that produces 48 output channels. While the channel count is appropriate, the stride-2 downsampling is counterproductive for small 32×32 images. Consequently, the following modification is applied:

- `features[0][0]` was replaced with a new `Conv2d(in_channels=3, out_channels=48, kernel_size=3, stride=1, padding=1, bias=False)`, preserving the channel count of the original stem while removing the spatial downsampling step.

The classification head was also replaced to match the target task: a dropout layer (default rate 0.5, configurable) was placed before a linear output layer adjusted to work for 10 classes.

2.2.3 Xception

The Xception architecture utilises depthwise separable convolution, effectively decoupling spatial filtering from channel mixing. The implementation follows a three-flow structure as specified in the original paper [1].

- **Entry:** The input is processed by two initial 3×3 convolutions (32 and 64 channels), followed by three residual blocks. These blocks utilize `SeparableConv2d` layers comprising depthwise and 1×1 pointwise convolutions, and strided max-pooling to increase channel depth from 64 to 728 while reducing spatial dimensions. Shortcut connections use 1×1 convolutions to match dimension changes.
- **Middle:** This stage consists of eight identical residual blocks operating on 728-channel feature maps at stride 1. These blocks refine learned representations through repeated depthwise separable convolutions without further spatial downsampling.
- **Exit:** A final residual block transitions the features to 1024 channels, followed by two `SeparableConv2d` layers reaching 2048 channels. Global average pooling (GAP) collapses the spatial dimensions into a 2048-dimensional vector, which is passed through a dropout layer (rate 0.5) and a linear layer for final classification.

Convolutional weights follow Kaiming normal initialization, while the linear head uses Xavier uniform. Batch normalization weights and biases are set to 1 and 0, respectively. The resulting architecture contains approximately 23 million parameters.

2.3 Experiments – Part 1: Hyperparameter Tuning

The first set of experiments aims to identify well-performing hyperparameter configurations for each architecture under standard supervised training. A grid of hyperparameters is evaluated, and the best configuration per architecture is selected for subsequent few-shot and ensembling experiments. All models are initialised with random weights and trained on CINIC-10. The hyperparameter grid is defined in Table 2 below.

Hyperparameter / Setting	Values Explored
Batch size	32, 64, 128, 256
Learning rate	0.1, 0.01, 0.001 (all with cosine decay)
Optimizer	SGD (momentum 0.9), Adam (default $\beta_1 = 0.9$, $\beta_2 = 0.999$)
L1 / L2 regularisation	Lasso (L1) and Ridge (L2) penalties with $\lambda = 0.001$
Dropout	Rates: 0.3, 0.5 (applied before the final classification head)

Table 2: Hyperparameter search grid.

All experiments are driven by a YAML-based `TrainingConfig` to ensure full reproducibility. The following protocol is applied across all training runs:

- **Loss and Metrics:** Models are optimised using cross-entropy loss (`nn.CrossEntropyLoss`). Performance is tracked via top-1 accuracy on the validation split at the end of every epoch.
- **Reproducibility:** A global seed (default: 42) is applied via `seed_everything()` to ensure deterministic behavior across PyTorch, NumPy, and Python’s random state.

- **Optimization:** Training utilises either SGD or Adam.
- **Early Stopping:** Training terminates if validation loss fails to improve for **patience** (default=5) consecutive epochs. Upon triggering, the best weights (lowest validation loss) are restored. Additionally, when the loss exceeds 10000 the training is stopped. This was introduced to terminate training process of models that did not promise convergence.

Due to computational limitations, the impact of data augmentation is not explored within this standard supervised setting; however, it will be investigated as a core component of the few-shot learning experiments.

The first stage of the hyperparameter tuning process involved evaluating configurations for batch size, learning rate, and the optimizer. While exhaustive testing was conducted for batch sizes of 64 and 128, a batch size of 256 exceeded available memory capacity, resulting in significantly degraded training efficiency. Thus only a few combinations of parameters were run (4 combinations for ResNet50 and 2 for EfficientNetB4). Consequently, only a limited subset of configurations was completed – four for ResNet50 and two for EfficientNetB4 – before further runs at this scale were discontinued. Instead, a batch size of 32 was evaluated, with a single run performed per model for both Adam with learning rate 0.001 and SGD with learning rate 0.01.

The results from this stage established the baseline parameters for the regularisation tuning phase. To maintain computational feasibility, only the best configuration – identified by the highest validation accuracy for each model – was selected for the experiments. A limit of 30 epochs was set for all experiments. However, the actual training duration was typically shorter due to the early stopping.

2.4 Experiments – Part 2: Few-Shot Learning with Contrastive Pretraining

2.4.1 Method Description

The few-shot learning pipeline has been expanded to a three-stage process to leverage synthetic data generation: (0) dataset augmentation, (1) supervised contrastive pretraining of the backbone, and (2) prototype-based classification using the pretrained embeddings.

Stage 0 – Dataset Augmentation. Given an initial set of k labeled training samples per class, an additional l samples are generated using specified augmentation techniques. To optimise computational resources, this augmented dataset generation is executed only once and saved for reuse across all subsequent training runs. To ensure compatibility with the pretrained weights of selected architectures, original images were upsampled accordingly upon the augmentation process. The augmentations are categorized into two types:

- **Standard:** Random rotations, horizontal/vertical flips, Gaussian noise, and Gaussian blur.
- **Advanced:** Fragment-wise augmentation via a custom pipeline using the custom Augmentor library [2]. Example images are shown in Figure 2.

This process yields a combined dataset of $k+l$ training samples per class to be used in the pretraining phase.



Figure 2: Images augmented with Augmentor library

Stage 1 – Contrastive Pretraining. Each of the three architectures (ResNet-50, EfficientNet-B4, Xception), initialised with ImageNet weights, is fine-tuned using a supervised contrastive objective on the full combined dataset of $k + l$ images per class. For each training batch, pairs of images from the same class are treated as positive pairs, while pairs of images from different classes are treated as negative pairs. The model is trained to produce embeddings such that positive pairs are clustered together and negative pairs are pushed apart in the embedding space. The training procedure follows the standard supervised contrastive loss formulation.

Stage 2 – Prototypical Network Classification. Following the contrastive pretraining stage, the trained encoder is used as a frozen embedding function within a prototypical network framework. Crucially, a class prototype is computed as the mean embedding vector over *only the k original* training images belonging to that class; the l augmented samples are excluded from this calculation to ensure the prototype represents the true data distribution. A query image is classified by computing the Euclidean distance between its embedding and each class prototype and assigning it to the nearest class. No additional parameters are trained in this stage.

This approach builds upon the prototypical networks framework of Snell et al. (2017) [10] and the contrastive representation learning approach reviewed in Hu et al. (2024) [7], while introducing targeted synthetic variety to improve encoder robustness.

2.4.2 Planned Experiments

The experimental design is structured to isolate the effects of the augmentation type, the volume of synthetic data (l), and the initial availability of true samples (k). All evaluations will report accuracy on the CINIC-10 validation split. The investigation is divided into three analytical phases:

Phase 1: Augmentation Type Impact

For a fixed baseline of original samples ($k = 10$) and a specified number of extra generated samples ($l = 50$), we will evaluate how the chosen augmentation technique (Standard variations vs. Advanced fragment-wise) impacts the overall classification accuracy. From these results, we will identify the top performing augmentations.

Due to limited computational resources only one of the architectures will be tested throughout phases 2-3

Phase 2: Scaling Augmented Samples (l)

Using only the top performing augmentations and top performing model architecture identified in Phase 1, we will systematically vary the number of extra samples (l) to investigate how the volume of augmented data impacts the overall accuracy.

Phase 3: Varying Original Samples (k)

Finally, we will investigate how the dynamics observed in Phases 1 and 2 shift depending on the initial number of original labelled samples (k). The following baseline conditions for k will be evaluated:

- **baseline:** For $k \in \{5, 10, 15, 20, 30, 40, 50\}$ prototypes are computed from embeddings coming from pretrained model weights with no additional training.
- **few-shot:** for $k \in \{5, 10, 15, 20, 30, 40, 50\}$ original labelled CINIC-10 samples per class, best performing architecture is trained using best l value from Phase 2.

2.5 Experiments – Part 3: Model Ensembling

Ensembling combines the predictions of multiple independently trained models to produce a final prediction that is typically more robust and accurate than any individual model. Two ensembling strategies are evaluated:

- **Hard voting:** each model casts a vote for its top-1 predicted class; the final prediction is the majority class. In case of a tie, the class with the highest cumulative probability is selected. Soft voting is used as a fallback mechanism in case of a tie.
- **Soft voting:** the softmax output probability vectors of all models are summed, and the class with the highest total probability is selected as the final prediction. For the few-shot learning protocol, the softmax is applied to the temperature-scaled cosine similarities between the query embeddings and each class prototype.

Both strategies are applied to the following sets of ensemble members:

1. All three architectures trained under the standard supervised protocol (Part 1, no regularisation).
2. All three architectures trained under the contrastive few-shot protocol (Part 2, all phases).
3. Pairwise combinations of architectures (ResNet-50 + EfficientNet-B4, ResNet-50 + Xception, EfficientNet-B4 + Xception) for both protocols.

3 Results Analysis

3.1 Individual Model Performance

During the initial experimental phase, seven configurations failed to converge due to exploding loss function values. Six of these instances occurred with the EfficientNet architecture:

- Adam, $LR = 0.1$, $BS = 64$
- SGD, $LR = 0.01$, $BS = 64$
- SGD, $LR = 0.001$, $BS = 64$
- Adam, $LR = 0.1$, $BS = 128$
- Adam, $LR = 0.01$, $BS = 128$
- SGD, $LR = 0.001$, $BS = 128$

Additionally, one ResNet configuration ($BS = 64$, Adam, $LR = 0.1$) failed to converge. While these cases were originally intended for further testing with regularisation techniques to evaluate potential stability improvements, they were ultimately excluded due to computational resource constraints.

The results for the best 5 and worst 5 performing configurations presented in Table 3 demonstrate that EfficientNetB4 turned out to be the superior architecture for this task, occupying four of the top five positions, specifically when paired with the Adam optimiser and a learning rate of 0.001. In contrast, a high learning rate of 0.1 with Adam often results in validation accuracies equivalent to random guessing (10%) or significantly degraded performance (25%) or, as mentioned before, lack of convergence.

Table 3: Top 5 and Bottom 5 Performing Model Configurations by Test Accuracy

Model Name	Batch Size	Optimizer	LR	Test Accuracy	Test F1-Score
<i>Best Performing Configurations</i>					
EfficientNetB4	32	Adam	0.001	0.8119	0.8121
EfficientNetB4	64	Adam	0.001	0.8066	0.8069
EfficientNetB4	128	Adam	0.001	0.7894	0.7906
ResNet50	64	Adam	0.001	0.7894	0.7895
EfficientNetB4	256	Adam	0.010	0.7874	0.7864
<i>Worst Performing Configurations</i>					
Xception	128	SGD	0.001	0.5969	0.5910
ResNet50	128	Adam	0.100	0.2498	0.1982
ResNet50	256	Adam	0.100	0.2497	0.2069
Xception	64	Adam	0.100	0.1000	0.0182
Xception	128	Adam	0.100	0.1000	0.0182

The training loss comparison in Figure 3 highlights Adam optimizer with $LR = 0.001$ as the most efficient setup, often reaching a stable plateau and triggering early stopping before epoch 20. Generally, Adam converges faster than SGD when paired with an appropriate learning rate, though it often fails to converge at $LR = 0.1$ as can be seen in 3. In contrast, SGD is more robust at higher rates, with $LR = 0.01$ yielding the best results for that optimizer. In terms of training time, EfficientNetB4 tended to utilise the most epochs, while ResNet50 was the quickest to converge, never requiring the full 30-epoch training window. To improve visualisation clarity, certain configurations were excluded to improve plots readability. These included ResNet50 (Adam, $LR = 0.1, BS \in \{128, 256\}$) and Xception (Adam, $LR = 0.1$), both of which exhibited high loss spikes or stagnant high-loss trajectories.

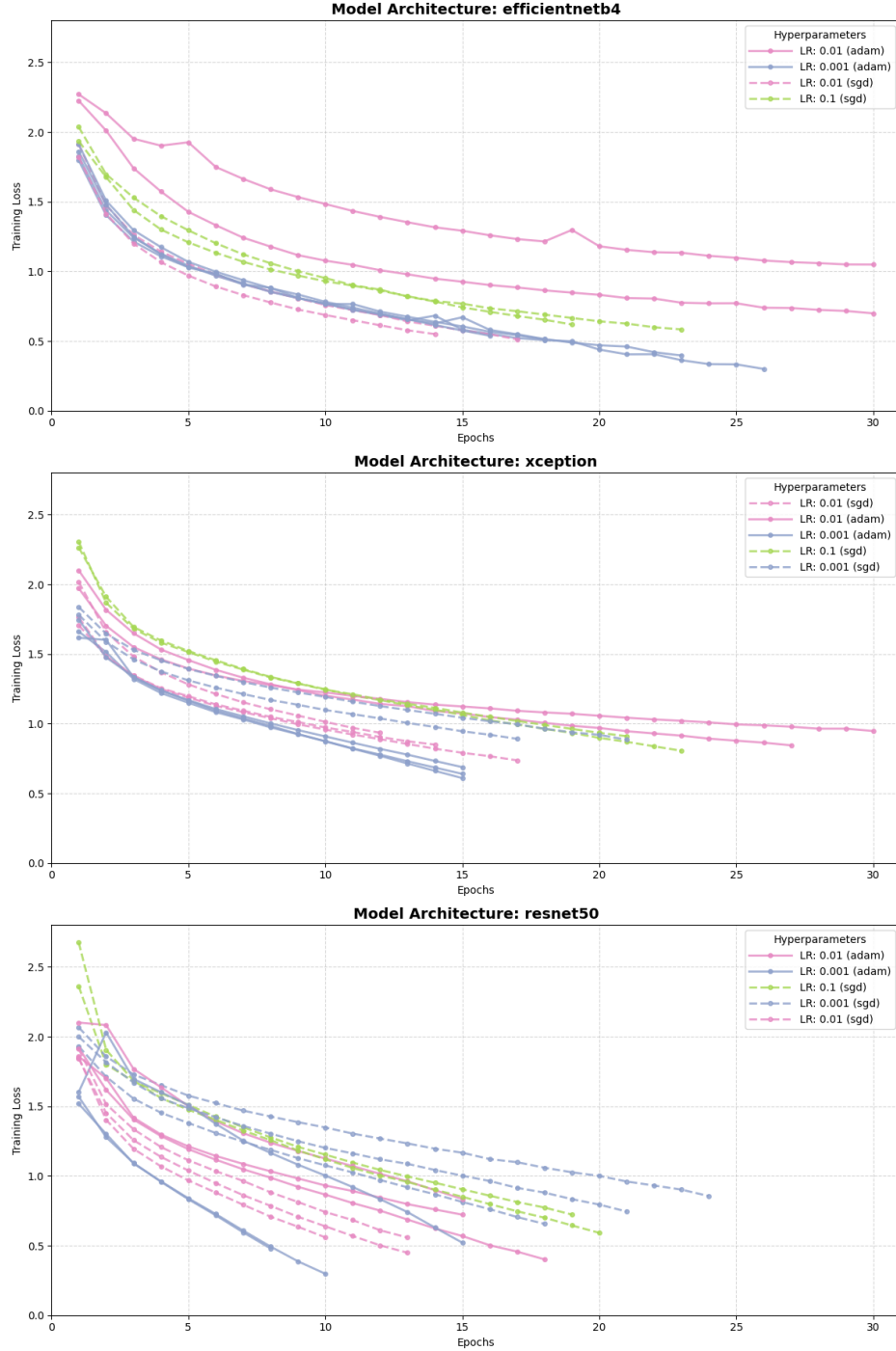


Figure 3: Training loss over epochs

Looking at the relationship between learning rate and batch size in Figure 4 , we can see that batch size mainly affects performance at lower learning rates, which generally produce weaker results. For a learning rate of 0.01, batch sizes of 32 and 256 yield the best performance, although the differences are relatively small. This outcome may be influenced by the experimental setup: a batch size of 2 was evaluated only once per model (using the best learning rate for each optimizer), and a batch size of 256 was used in just 6 out of 36 runs, as noted in the experiment description. This imbalance could have affected the results. For a learning rate of 0.001, smaller batch sizes tend to perform better. Overall, a batch size of 128 provides the

most stable results, showing the least variation in test accuracy.

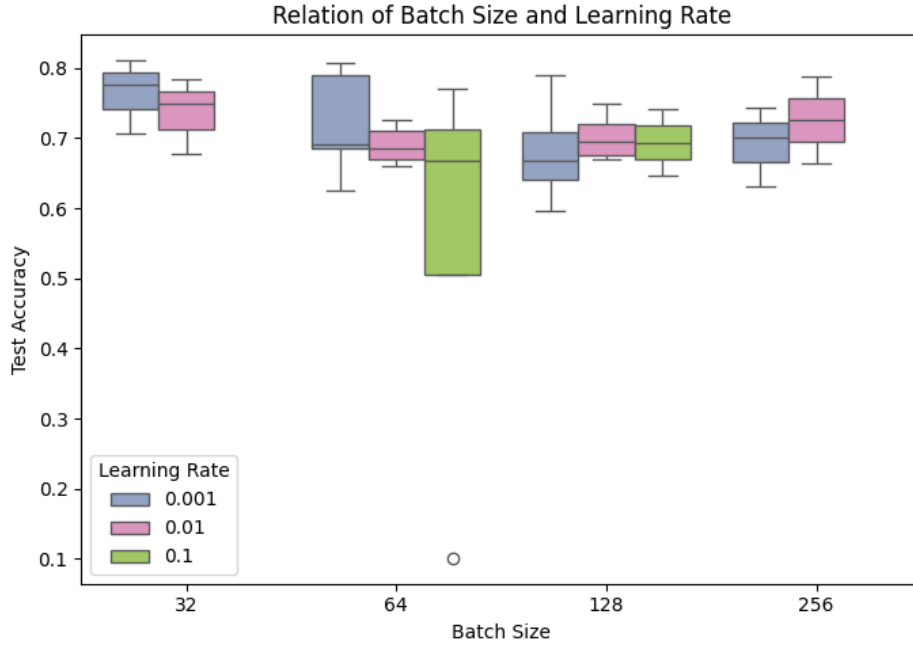


Figure 4: Relation between learning rate and batch size

Based on the results obtained in phase 1, the following baseline models were selected for the regularisation experiments:

- EfficientNetB4: Adam, $LR = 0.001$, $BS = 64$
- ResNet50: Adam, $LR = 0.001$, $BS = 64$
- Xception: Adam, $LR = 0.001$, $BS = 32$

Figure 5 summarises the impact of different regularisation strategies on test accuracy. The application of dropout resulted in negligible changes compared to the baseline models without regularisation. In contrast, both L1 and L2 regularisation led to a noticeable decline in performance across most configurations. As generally expected, L1 regularisation tended to have worse effect than L2, which was consistent for both Xception and ResNet50. However, an opposite trend was observed for EfficientNetB4, where L2 appeared to have a stronger negative impact. Given that each configuration was evaluated only once, this observation should be treated with caution, as it may not reflect a statistically robust effect.

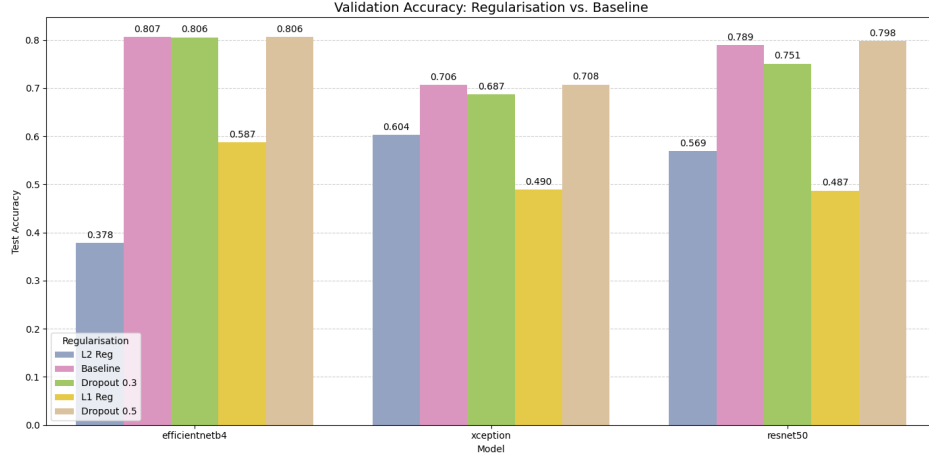


Figure 5: Regularisation influence on accuracy

3.2 Few-Shot Learning Results

Phase 1

Three ImageNet-pretrained convolutional network architectures (ResNet50, EfficientNet-B4, and Xception) were modified by replacing the classification head with a projection head, which mapped high-dimensional features into a 128-dimensional latent space. Then last 3 blocks of each of the models were trained on the same subset of 10 original and 50 augmented images per class. Training was conducted using Supervised Contrastive loss with a temperature parameter of 0.07. This loss uses cosine similarity, which is used to pull the embeddings of images of the same type together while maximising the distance between different classes. The number of epochs was set to 20 throughout the experiment and the Adam optimiser was used. After the training phase, prototypes of each class were calculated as a mean value of original training data samples. Then, the models were evaluated on the validation dataset by measuring cosine similarity between the validation images and the previously computed prototypes. Accuracy results for each of the models and augmentation methods are presented in the Figure 6.

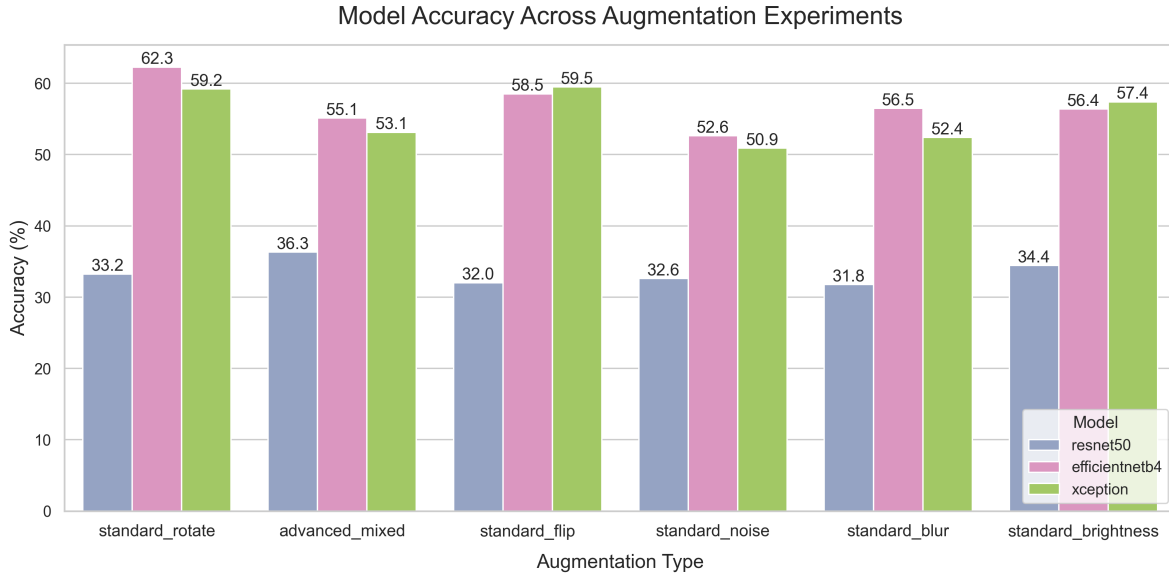


Figure 6: Results of Phase 1 Experiments

What can be observed is that ResNet architecture performed the worst, while Xception and EfficientNet achieved similar results. The best performing augmentation were rotations. The following augmentation methods were chosen for the next phase:

- rotate
- a combination of rotate, flip and noise
- custom Augmentor segment-wise augmentation.

Due to limited computational resources availability, only the EfficientNet-B4 model was used in phases 2 and 3 of the experiment, and a 10% subset of the evaluation dataset was selected for testing.

Phase 2

Phase 2 builds upon the methodology established in Phase 1 to investigate how the number of augmented images influences the model predictions quality. The training process was repeated with a variable number of augmented samples, defined by the parameter $l \in \{0, 5, 15, 30, 50\}$, where l corresponds to the number of augmented images per class. The results are shown in Figure 7.

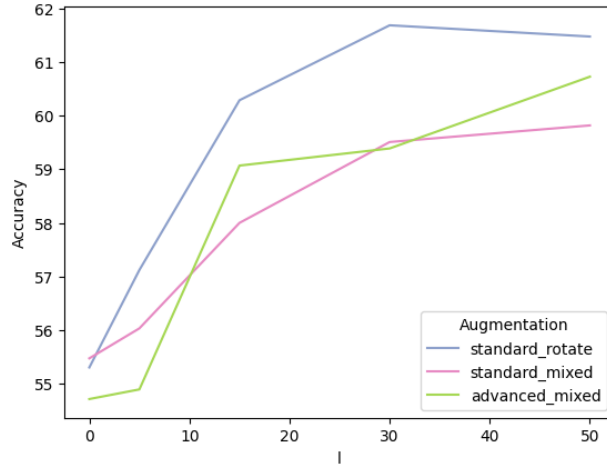


Figure 7: Results of Phase 2 Experiments

Performance of the model improves as the l rises for all the augmentation scenarios. Interestingly, a decrease in accuracy appears only for rotation augmentation style, for $l = 50$, which may point to overfitting. The most significant improvements are observed for smaller l values with performance beginning to plateau as l increases. This demonstrates that image augmentation is a viable approach for addressing few-shot learning challenges. In this experimental setup, rotation appeared to be the most effective strategy, though its advantage over other methods was not significant.

Phase 3

In Phase 3, the experiment focuses on the influence of original training samples used for training ($k \in \{5, 10, 15, 20, 30, 40, 50\}$). To provide a point of comparison, a baseline evaluation was conducted using the EfficientNet and Xception models in a prototypical network setting without any additional training. k values were used only for prototypes calculation. Results of baseline experiment are shown in Figure 8.

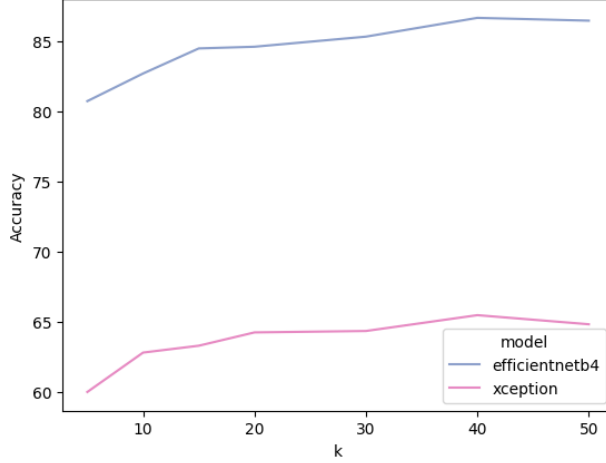


Figure 8: Results of Baseline Models

A strong correlation between k parameter and model accuracy was observed. Intuitively, the more samples are used to calculate the class prototypes, the more accurate the model becomes. It is worth noticing, that the results of the baseline models (before applying the contrastive learning) are significantly better than the previously discussed, contrastive-trained ones. This suggests that the ImageNet weights already possess a near-optimal representation of the CINIC-10 domain, and the computational overhead of supervised contrastive pretraining provides diminishing returns. Consequently, for Phase 3, $l = 0$ was selected as the operational standard. This decision was made to prioritise computational efficiency and to isolate the impact of the support set size (k) without the confounding influence of synthetic data.

The results for the EfficientNet-B4 architecture pretrained on subsets where $k \in \{5, 10, 15, 20, 30, 40, 50\}$ are shown in Figure 9. It can be observed that accuracy correlates positively with k parameter, like the baseline model. However, results of the contrastive trained model are much worse.

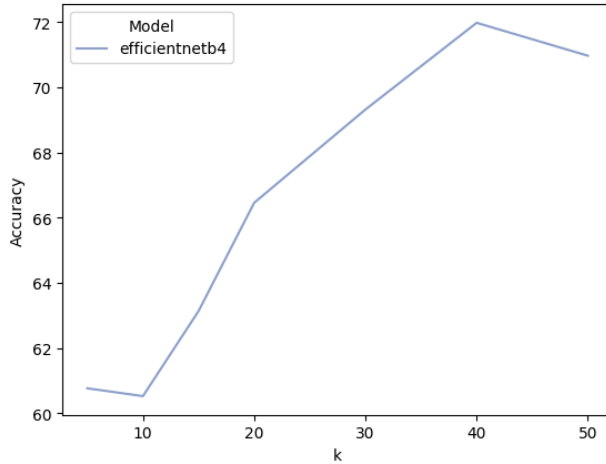


Figure 9: Results of Phase 3 Experiment

As shown, contrastive learning proved unnecessary on the architectures pretrained on ImageNet dataset. This can be a result of a data leakage between ImageNet and CINIC-10 datasets. The contrastive learning might overfit the model for specific subset of images, worsening the performance overall. It is proposed that in further research contrastive few-shot learning could be considered on architectures pretrained on different datasets or when training models from scratch. In the current experimental configuration, the

added complexity of contrastive learning appears to be redundant.

3.3 Ensemble Results

As in the previous section, in the ensemble experiments all models were evaluated on a 900 image validation subset for computational optimisation reasons. All experiments related to few-shot learning were performed on k parameter set to 10 (images per class, necessary to compute class prototypes).

Part 1 – Standard Training Protocol

For these experiments, individual models were trained using a standard approach, detailed in Section 2.3. The results do not include regularised models, as the calculations were computed simultaneously on different machines. Top 1 configuration of each architecture was chosen, and the predictions were combined using standard methods. Results are shown in Figure 10 below. In this scenario, we don’t observe a significant difference between the soft and hard voting strategies (less than 1% difference in all cases), but the overall improvement in accuracy with regard to the best individual model remains (increase of approximately 8-12%).

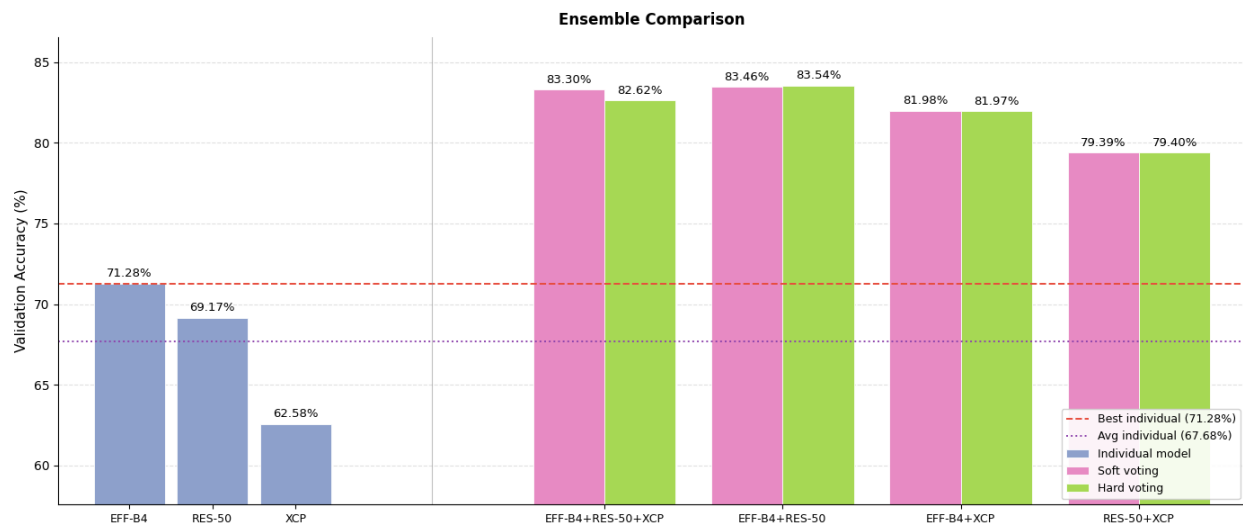
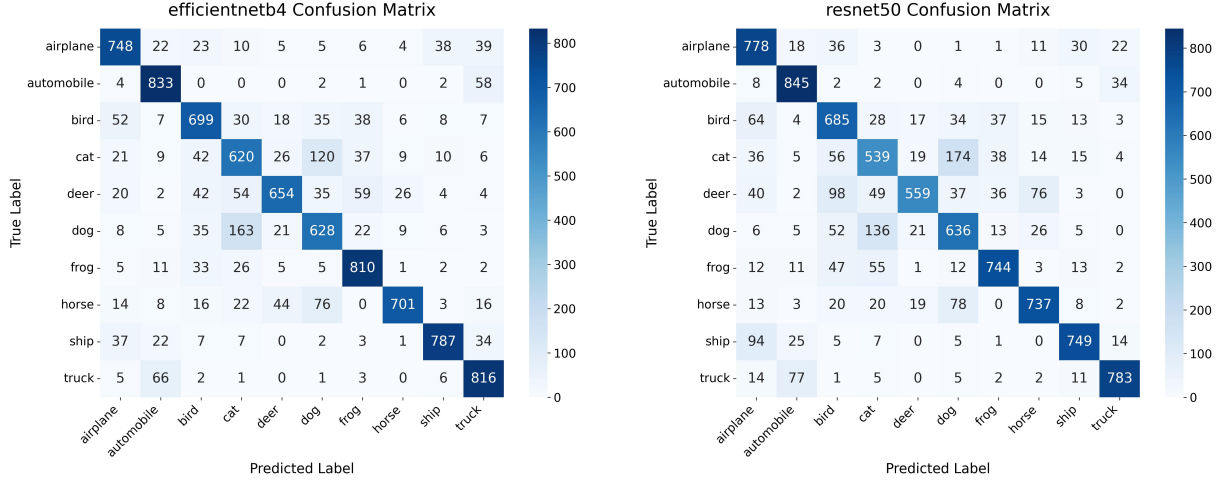


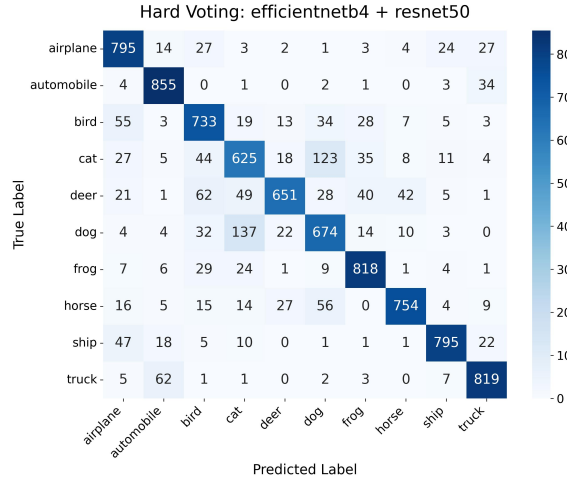
Figure 10: Individual Model Accuracy vs. Ensemble Performance - Part 1 (standard training, different models)

In order to investigate the cause of this increase in performance, we took a closer look at confusion matrices of individual models included in the best ensemble configuration: EfficientNet-B4 (Figure 11a) and ResNet-50 (Figure 11b), and compare them with the ensemble results (Figure 11c). All three matrices show that the most commonly misclassified classes were cats and dogs (both ways), with over 100 images misclassified in each case. This is not surprising given the similarity between the species and the small resolution of selected images. In each case, ‘automobile’ was least often misclassified, along with ‘truck’ taking the second place in 1-vs.-all accuracy. An interesting observation can be made when looking at the number of ‘horse’ images misclassified as ‘dog’. Despite both individual modes being wrong on nearly 80 instances, this number falls down below 60 in the ensemble. This could point to effectiveness of the fallback mechanism of using soft voting in case of a tie instead of a random guess – this idea could be further investigated.



(a) EfficientNet-B4: 71.28% Acc

(b) ResNet-50: 69.17% Acc



(c) top ensemble: 83.54% Acc

Figure 11: Confusion matrices of the top performing ensemble configuration (on 900 image validation subset)

Part 2a – Few-Shot Learning, Phase 1 (different architectures)

For these experiments, individual models were trained using a contrastive learning approach with different augmentation techniques, as detailed in Sections 2.4.2 and 3.2. Again, top 1 configuration of each architecture was chosen and then all of their predictions were combined. Results are shown in Figure 12 below. In this setting, differences between soft and hard voting techniques are much more defined. This is most likely due to the ResNet-50 architecture negatively impacting the results. Despite the very low relative performance of that architecture – when combined with the other two – the results remain satisfactory since the two remaining models can easily have the majority vote. In the pairwise setting, the negative impact is significantly higher, especially when hard voting, which could suggest lower confidence of the ResNet-50 model, what would be consistent with its overall low performance.

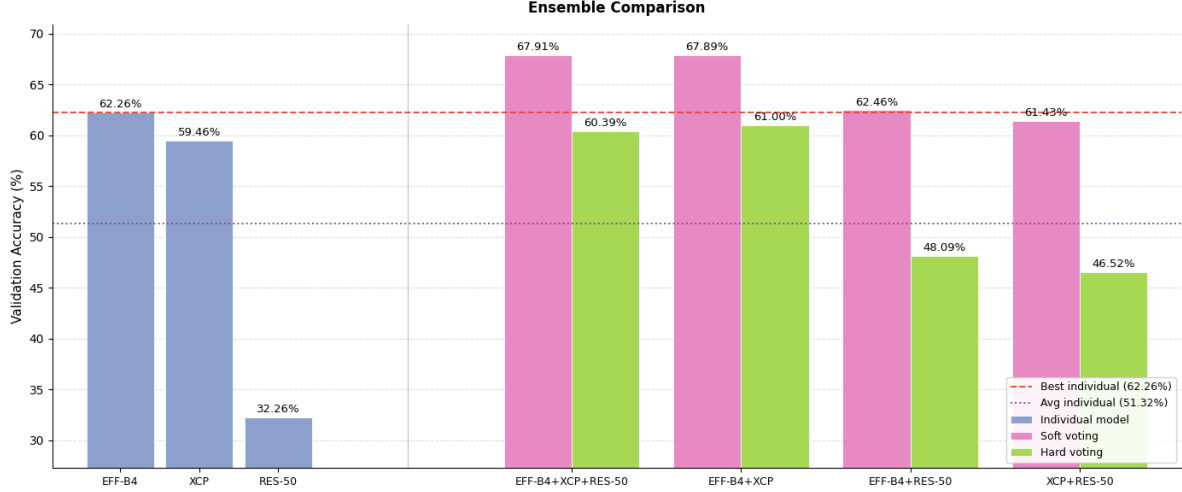


Figure 12: Individual Model Accuracy vs. Ensemble Performance - Part 2a (different architectures)

Part 2b – Few-Shot Learning, Phase 2 (same architecture, different l parameter)

In these experiments, a single pretrained architecture was fine-tuned using a contrastive learning approach with different numbers of augmented training samples l , as detailed in Sections 2.4.2 and 3.2 (Phase 2). Top 3 configurations of the EfficientNet-B4 architecture were chosen for ensembling. Results are shown in Figure 13 below. In this setting, the difference between performance of individual models is the smallest, but the difference between soft and hard voting settings remains significant. Interestingly, we observe a decrease in performance in two of the combinations in the hard voting setting when compared with the poorest result of the three members. Therefore, it is important to consider that it is not guaranteed that ensemble results will always be superior, especially when using different versions of the same architecture.

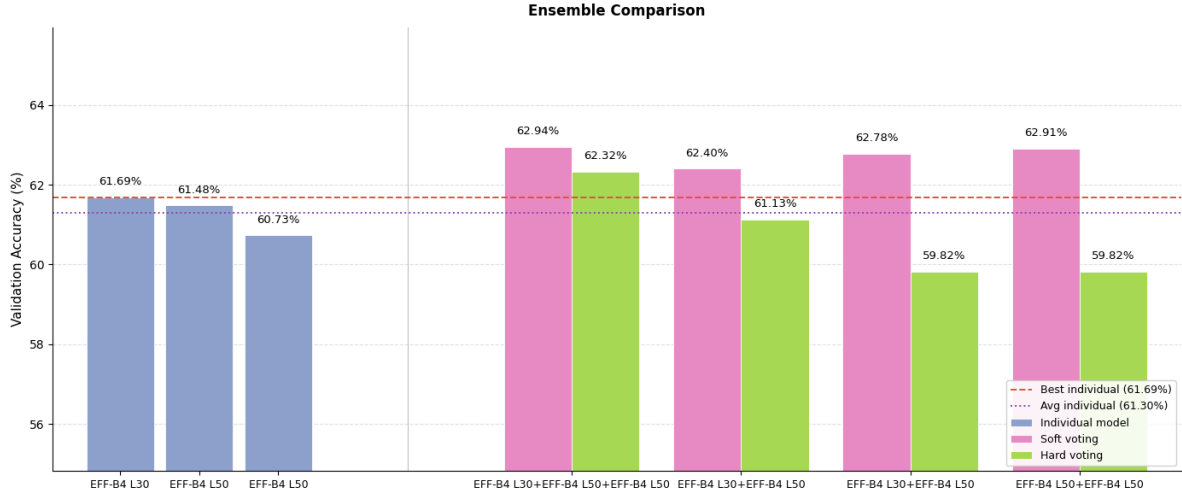


Figure 13: Individual Model Accuracy vs. Ensemble Performance - Part 2b (same architecture, different l parameter)

Part 2c – Few-Shot Learning, Phase 3 (transfer learning, no additional training)

In these experiments, all pretrained architectures were input directly into the protonet setup with no additional training (baseline), as detailed in Sections 2.4.2 and 3.2 (Phase 3). As seen in Figure 14, soft voting generally outperforms hard voting, but this time none of those configurations managed to beat the maximum

individual scores. It seems that the EfficientNet-B4 model, despite good individual performance, was not confident enough in its predictions to outvote its co-memebers.

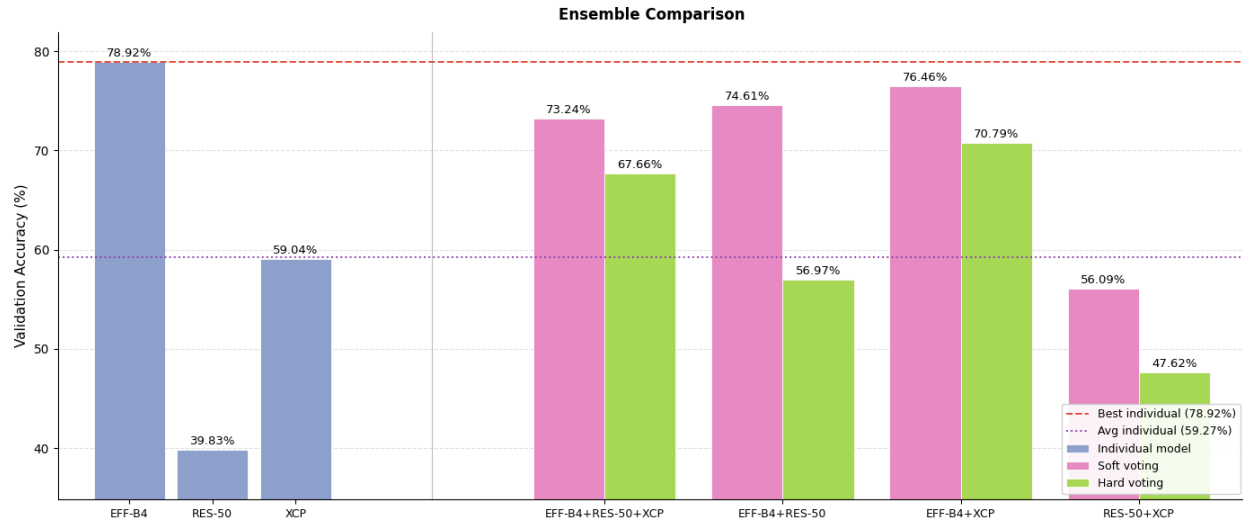


Figure 14: Individual Model Accuracy vs. Ensemble Performance - Part 2c (transfer learning, no additional training)

Summary

Ensemble Configuration	Max Ind.	Avg Ind. ($\pm$ Sd)	Soft Vote	Hard Vote
Part 1 - Standard Training Protocol				
efficientnetb4 + resnet50 + xception	71.28%	67.68% $\pm$ 3.70	83.30%	82.62%
efficientnetb4 + resnet50	71.28%	70.22% $\pm$ 1.06	83.46%	83.54%
efficientnetb4 + xception	71.28%	66.93% $\pm$ 4.35	81.98%	81.97%
resnet50 + xception	69.17%	65.87% $\pm$ 3.29	79.39%	79.40%
Part 2a - Few-Shot Learning, Phase 1 (different architectures)				
efficientnetb4 + xception + resnet50	62.26%	51.32% $\pm$ 13.53	67.91%	60.39%
efficientnetb4 + xception	62.26%	60.86% $\pm$ 1.40	67.89%	61.00%
efficientnetb4 + resnet50	62.26%	47.26% $\pm$ 15.00	62.46%	48.09%
xception + resnet50	59.46%	45.86% $\pm$ 13.60	61.43%	46.52%
Part 2b - Few-Shot Learning, Phase 2 (same arch., different l)				
eff_l30_s_rot + eff_l50_s_rot + eff_l50_a_mix	61.69%	61.30% $\pm$ 0.41	62.94%	62.32%
eff_l30_s_rot + eff_l50_s_rot	61.69%	61.58% $\pm$ 0.11	62.40%	61.13%
eff_l30_s_rot + eff_l50_a_mix	61.69%	61.21% $\pm$ 0.48	62.78%	59.82%
eff_l50_s_rot + eff_l50_a_mix	61.48%	61.11% $\pm$ 0.37	62.91%	59.82%
Part 2c - Few-Shot Learning, Phase 3 (transfer learning - no additional training)				
efficientnetb4 + resnet50 + xception	78.92%	59.27% $\pm$ 15.96	73.24%	67.66%
efficientnetb4 + resnet50	78.92%	59.38% $\pm$ 19.54	74.61%	56.97%
efficientnetb4 + xception	78.92%	68.98% $\pm$ 9.94	76.46%	70.79%
resnet50 + xception	59.04%	49.44% $\pm$ 9.61	56.09%	47.62%

Table 4: Ensemble results across all experimental protocols comparing maximum, average, and standard deviation of individual model accuracies to soft and hard voting strategies.

Table 4 summarises all experimental results. The overall best performing single model was a pre-trained EfficientNetB4 used as a backbone in a prototypical network setup, achieving a score of 79.82% on the validation set. The architecture also performed best in case of the models trained from scratch (71.28% accuracy). It consistently outperformed the other two models, when considering best configurations for each architecture. The highest score achieved by an ensemble was 83.54% (hard voting, standard training setup). Aside from the standard training setup, where differences in scores of soft and hard voting ensembles are minimal, the soft voting strategy consistently outperforms the other.

4 Conclusions

In the standard supervised training experiments on the CINIC-10 dataset, EfficientNet-B4 emerged as the superior architecture, achieving a top individual accuracy of 71.28% when trained from scratch. The hyperparameter configuration significantly impacted these results, with the Adam optimiser paired with a learning rate of 0.001 proving to be the most effective choice. Conversely, applying a high learning rate of 0.1 with Adam led to a failure to converge or yielded performance equivalent to random guessing. Regarding batch sizes, a size of 128 provided the most stable results with the least variation in test accuracy. Furthermore, standard regularization techniques like L1 and L2 penalties resulted in a noticeable decline in model performance, while dropout showed negligible effects compared to the unregularised baseline.

The contrastive few-shot learning approach ultimately proved unnecessary and redundant for architectures that were already pretrained on the ImageNet dataset. The baseline models, which were evaluated without any additional contrastive training, significantly outperformed the contrastive-trained models. This diminished performance likely resulted from data leakage between the ImageNet and CINIC-10 datasets, causing the contrastive learning process to overfit to specific image subsets and worsen overall performance. Despite this, the experiments demonstrated that model accuracy scales positively with an increase in both the number of original training samples used for prototypes and the number of augmented images. However, the performance gains from adding synthetic data plateaued as the volume of augmented images increased.

Model ensembling provided substantial performance gains, improving accuracy by approximately 8% to 12% over the best individual model under the standard training protocol. The most effective ensemble combination utilised a hard voting strategy with the EfficientNet-B4 and ResNet-50 architectures, achieving a peak accuracy of 83.54%. While the difference between soft and hard voting strategies was minimal in the standard training setup, soft voting consistently outperformed hard voting across the few-shot learning experiments.

The primary limitations of this study stemmed from severe computational resource constraints. These limitations forced a prioritization of experimental breadth over statistical significance, meaning configurations were generally evaluated without multiple runs. Additionally, these constraints necessitated the use of a reduced 10% subset of the validation and test datasets during the experiments to save time and memory. For future work, it is recommended to explore contrastive few-shot learning on architectures pretrained on entirely different datasets, or to train the models from scratch, to mitigate the negative effects of data leakage. Furthermore, further investigating the effectiveness of using soft voting as a fallback mechanism for tie-breaking in hard voting scenarios presents a promising direction for future research.

References

- [1] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1800–1807, 2017.
- [2] Natalia Choszczyk, Mateusz Deptuch, Karolina Dunal, and Zofia Kamińska. Losowa segmentacja obrazów i augmentacja lokalna jako strategia poprawy uczenia głębokiego, 04 2025. Attachment 1.
- [3] Luke Darlow, Elliot Crowley, Antreas Antoniou, and Amos Storkey. Cinic-10 is not imagenet or cifar-10, 10 2018.
- [4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [6] IBM. What is few-shot learning?, 2024.
- [7] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA, 2020. Curran Associates Inc.
- [8] Alex Krizhevsky. Learning multiple layers of features from tiny images. *University of Toronto*, 05 2012.
- [9] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [10] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, page 4080–4090, Red Hook, NY, USA, 2017. Curran Associates Inc.
- [11] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks, 05 2019.

5 Appendix

Reproducibility and Technical Requirements

This appendix outlines the repository structure, environment setup, and execution steps required to replicate the experiments, including standard supervised training, few-shot contrastive pretraining, and model ensembling on the CINIC-10 dataset.

Repository Structure

The project codebase is organized to separate executable scripts, configuration parameters, modular deep learning components, and saved model states. The repository structure is as follows:

- **README.md**: High-level overview of the project and quick-start guide.
- **.gitignore**: Specifies intentionally untracked files to ignore (e.g., datasets, large **.pth** files, **__pycache__**).
- **requirements.txt**: Complete list of Python package dependencies required to run the environment.
- **data/**: Full dataset downloaded from kaggle.
 - **train/**: original dataset
 - **valid/**: original dataset
 - **test/**: original dataset
 - **augmented_experiments/**: Contains augmented data prepared for Part 2 of the experiments.
- **pretrained_encoders_contrastive/**: Directory containing final trained model weights from the few-shot learning setup (**.pth** files), organized by architecture and augmented dataset version.
- **saved_models/**: Directory containing final trained model weights (**.pth** files), organized by architecture:
 - **resnet50/**
 - **efficientnetb4/**
 - **xception/**
- **code/**: Main directory containing all source code for the experiments.
 - **train.py**: The primary execution script for orchestrating the training and evaluation loops.
 - **utils.py**: Helper functions for tasks such as setting global random seeds (**seed_everything()**), checkpointing, and metric logging.
 - **models/**: Contains the PyTorch implementations of the architectures.
 - * **resnet50.py**, **efficientnetb4.py**, **xception.py**
 - **configs/**: YAML-based configuration files for managing experiment hyperparameters.
 - * **configs.py**: Python script for parsing and validating YAML parameters.
 - * **default.yaml**: Baseline hyperparameter settings for standard supervised training.
 - * **augmentor.yaml**: Specific configurations for the synthetic data generation and few-shot pipeline.
 - **augmentations/**: Modules handling data preprocessing and synthetic generation.
 - * **augmentor.py**: Custom fragment-wise augmentation implementation.
 - * **transforms.py**: Standard PyTorch/Torchvision transformations (rotations, flips, noise, etc.).
 - * **datasetwrapper.py**: Custom PyTorch Dataset classes to handle integration with the Augmentor class.

Environment Setup and Technical Requirements

To ensure full reproducibility across different machines, all experiments should be executed within an isolated Python environment (e.g., virtualenv or Conda).

1. **Python Version:** The codebase requires Python 3.8 or higher.
2. **Hardware Requirements:** Training deep CNNs like Xception and EfficientNet-B4 prefers a CUDA-enabled NVIDIA GPU with at least 8GB of VRAM. CPU execution is supported for inference but is not recommended for training.
3. **Dependencies:** Install all required libraries using the provided requirements file:

```
pip install -r requirements.txt
```

(Core dependencies include *PyTorch*, *Torchvision*, *NumPy*, *PyYAML*, *Scikit-learn*, *Pillow*).

Execution Instructions

All training and evaluation routines are executed via the centralized `train.py` script, controlled entirely by the YAML configuration files.

1. Standard Supervised Training (Part 1)

To replicate the hyperparameter tuning and standard baseline training, execute:

```
python code/train.py --config code/configs/default.yaml
```

Modifications to learning rates, batch sizes, optimizers, and target architectures should be adjusted directly within `default.yaml` prior to execution.

2. Few-Shot Learning and Contrastive Pretraining (Part 2)

To execute the data generation use `generate_dataset()` function from `code/few_shot_learning/few_shot_dataset.py`.

To execute Phase 1 run:

```
python code/few_shot_learning/few_shot_phase_1.py
```

```
python code/few_shot_learning/contrastive_evaluation_1.py
```

For Phase 2, use:

```
python code/few_shot_learning/few_shot_phase_2.py
```

```
python code/few_shot_learning/contrastive_evaluation_2.py
```

To execute Phase 3, run

```
python code/few_shot_learning/few_shot_phase_3.py,
```

```
python code/few_shot_learning/contrastive_evaluation_3.py
```

```
python code/few_shot_learning/baseline.py
```

3. Evaluating Pre-trained Models and Ensembles (Part 3)

To bypass training and evaluate the final models or run the hard/soft voting ensemble strategies, use the weights stored in the `saved_models/` directory for the standard approach and in `pretrained_models_contrastive/` for the few-shot learning setup. The evaluation mode can be triggered via:

```
python code/ensembles/ensembles.py,
```

for the standard setup, and

```
python code/ensembles/ensembles_FS.py,
```

for the few-shot learning setup. This triggers all configurations for both soft and hard voting. The number of members of each ensemble can be adjusted by using a *top-k* argument.

Fun Fact

While deep diving into confusion matrices of different models we found that the ResNet-50 architecture trained in our few-shot learning setup seems to be a dog person and hates cats. It only predicted the ‘cat’ class a total of 108 times (out of 900!), of which only 34 were correct, as seen in Figure 15.

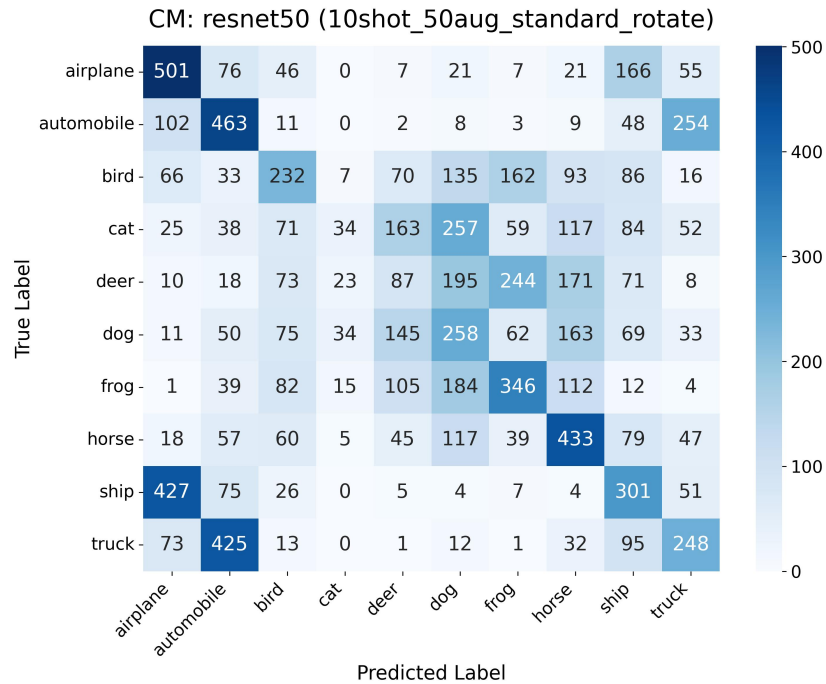


Figure 15: Cat Hater ResNet-50